

QuantumSimEvo: A Portable Quantum Computer State-Vector Simulator for Education and Commodity Hardware

Egor Knyazev
Fontys University of Applied Sciences

June 13, 2026

Abstract

Until large-scale fault-tolerant hardware arrives, classical state-vector simulation is how most people develop, debug, and learn quantum algorithms. Established simulators are fast, but using them often means a heavy dependency stack, a difficult build, or a particular accelerator—barriers that get in the way for a student on an ordinary laptop. We present QuantumSimEvo, a portable C++ state-vector simulator for teaching and low-friction experimentation on a commodity PC: it runs correctly anywhere and transparently accelerates on whatever hardware the host provides, from a single dependency-free binary. We validate it on Bell and GHZ states and five algorithms—the Quantum Fourier Transform, Grover’s search, Shor’s factoring, and QAOA for MaxCut and the Travelling Salesman Problem. Benchmarked against Qiskit Aer, QuantumSimEvo matches that production framework to within roughly 12% on the CPU and stays within a small constant factor on the GPU, without external dependencies.

1 Introduction

Quantum computers promise asymptotic speedups for problems like integer factoring [1], unstructured search [2], and combinatorial optimization [3]. The hardware that exists today still has comparatively few qubits, is noisy, and is hard to get time on, so classical simulation does most of the everyday work of designing, debugging, and teaching quantum algo-

rithms. The most direct approach is *state-vector simulation*: store the full quantum state as 2^n complex amplitudes, where n is the number of qubits, and apply each gate as a structured transformation of that array—formally a matrix-vector multiplication. Because a simulator keeps the exact amplitudes in memory and can read them at any point—unlike a physical device, whose state collapses on measurement—it is well suited for verification and teaching. The price is that both memory and time grow exponentially in the qubit count n .

Exponential growth in computation time and memory motivated a generation of carefully optimized simulators. Production frameworks such as Qiskit Aer [4], Google’s Cirq [5], and ProjectQ [6] pair rich circuit-construction APIs with fast compiled backends, while QuEST [7], the Intel Quantum Simulator [8], and NVIDIA cuQuantum [9] pursue peak performance through multithreading, distributed memory, and GPU acceleration. All of them are capable, but the capability tends to come with strings attached: a large Python or C++ dependency stack, a fiddly build, or—for the fastest options—a particular accelerator, or access to a shared high-performance computing (HPC) cluster. For someone meeting quantum computing for the first time on a laptop, those barriers can get between the student and the concepts the simulator is meant to teach.

QuantumSimEvo starts from a different priority: a *portable, educational* state-vector simulator that runs on a commodity PC with no special hardware and almost no setup, yet still takes advantage of whatever

acceleration the host happens to provide. We refer throughout to the person driving the simulator as the *user*; in our setting the user is typically a student, since teaching is the primary use case. Three design priorities follow from that. The first is *portability over peak performance*—the simulator has to run correctly everywhere and accelerate only where it can, never depending on a feature a basic machine might lack. The second is a clean *separation between circuit semantics and execution*, which keeps the user-facing model simple and leaves the engine free to change underneath it. The third is *extensibility by addition*: a new execution backend, the GPU backend included, should slot in without disturbing code that already works.

This paper makes the following contributions:

- A **backend-pluggable architecture** (Section 3) that decouples quantum-circuit semantics from amplitude manipulation behind a single abstract operation interface, allowing interchangeable execution backends.
- A **runtime backend-selection** mechanism that probes the host and dispatches to the most capable implementation available, degrading gracefully from a CUDA GPU backend to AVX2 SIMD to a portable OpenMP backend—all from a single binary.
- A **validation suite** (Section 4) of entanglement primitives (Bell, GHZ) and five algorithms—QFT, Grover’s search algorithm [2], Shor’s factoring algorithm with a space-efficient $4n + 2$ formulation following Beauregard [10, 1], the Quantum Approximate Optimization Algorithm (QAOA) [3] for MaxCut, and a constraint-preserving variant based on the Quantum Alternating Operator Ansatz [11] for the Travelling Salesman Problem—implemented entirely through the public gate API.

The remainder of the paper is organized as follows. Section 3 presents the architecture and runtime backend selection; Section 4 details the algorithmic case studies; and the evaluation and concluding discussion follow.

2 Theory

This section collects just enough quantum-computing formalism to follow the rest of the paper, then connects it to *state-vector simulation* and to the cost that drives the engineering choices in Section 3. Readers who want the full treatment should consult Nielsen and Chuang [12].

2.1 Qubits and Quantum States

The basic unit of quantum information is the *qubit*, whose pure state is a unit vector in the two-dimensional complex Hilbert space $\mathbb{C}^2$,

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1, \quad (1)$$

with $\{|0\rangle, |1\rangle\}$ the computational basis and $\alpha, \beta \in \mathbb{C}$ probability amplitudes; the normalization $|\alpha|^2 + |\beta|^2 = 1$ ensures that the two measurement probabilities sum to one. A register of n qubits lives in the tensor-product space $(\mathbb{C}^2)^{\otimes n} \cong \mathbb{C}^{2^n}$, and a general state is the superposition

$$|\psi\rangle = \sum_{x=0}^{2^n-1} c_x |x\rangle, \quad \sum_{x=0}^{2^n-1} |c_x|^2 = 1, \quad (2)$$

where each basis state $|x\rangle$ is indexed by an n -bit string x . The 2^n coefficients c_x form the *state vector*, and this array is precisely what a state-vector simulator holds in memory. It is also where the difficulty comes from: a classical register of n bits needs n numbers, but the quantum register needs 2^n , so the memory footprint grows exponentially in n .

A state that cannot be factored into a tensor product of single-qubit states is *entangled*. The Bell and GHZ states of Section 4 are the textbook examples, and we use them as the simulator’s first correctness checks.

2.2 Gates and Circuits

Quantum computation proceeds by applying *gates*, which are unitary operators acting on one or a few qubits. An operator U is *unitary* when

$$U^\dagger U = I, \quad (3)$$

where $U^\dagger$ denotes the conjugate transpose (adjoint) of matrix U , and I is the identity matrix of the same size. A single-qubit gate is a 2×2 unitary matrix. Representative examples used throughout this work include the Hadamard gate,

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad (4)$$

the Pauli operators,

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad (5)$$

and the parameterized rotations, for instance about the z axis,

$$R_z(\theta) = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}. \quad (6)$$

The Hadamard (Eq. 4) creates superposition, the Pauli operators (Eq. 5) are the elementary bit- and phase-flips, and the rotations (Eq. 6) supply the continuous parameters that the variational algorithms of Section 4 tune. Multi-qubit gates can establish entanglement; the controlled-NOT (CNOT) and the multi-controlled phase (MCPhase) used by Grover’s and Shor’s algorithms are the ones we lean on most.

A *quantum circuit* is an ordered sequence of such gates applied to a register, usually ending in a measurement. A gate that touches only k qubits acts as the identity on the other $n - k$, so its effect on the full state vector is very regular and structured. The simulator’s kernels are built around exactly that structure (Section 3.2).

2.3 Measurement

Results are obtained by *measurement* in the computational basis (the basis of definite bit-string states $|x\rangle$). Measuring an n -qubit state returns outcome x with probability $P(x) = |c_x|^2$ and collapses the state to $|x\rangle$. A single run therefore yields one basis state $|x\rangle$, sampled from the distribution $\{|c_x|^2\}_x$, so estimating the full distribution means repeating the prepare-and-measure cycle over many *shots*. The simulator reproduces this by sampling repeatedly from the computed state and tallying the resulting outcomes.

2.4 State-Vector Simulation

State-vector simulation (also called full-amplitude or Schrödinger-style simulation [12]) keeps the quantum state explicitly, as an array of 2^n amplitudes c_x , and applies each gate as an in-place transformation of that array. Applying a single-qubit gate to qubit q comes down to walking the 2^{n-1} index pairs $(x, x \oplus 2^q)$ that differ only in bit q and applying the same 2×2 matrix to each pair; a controlled gate simply skips the pairs whose control bits are not set. Crucially, every pair is updated independently—each amplitude belongs to exactly one pair, and no pair reads or writes another pair’s entries—so the 2^{n-1} updates carry no data dependencies among them. The access pattern is therefore uniform, branch-free, and free of cross-element data dependencies: the pairs can be split across OpenMP threads, processed several at a time within AVX2 vector lanes, or mapped one per thread onto a GPU, all without synchronisation.

The payoff is exactness. Because it carries the whole amplitude vector, the method reproduces interference and entanglement with no approximation, which is what makes it good for verification and teaching. The cost is the exponential: 2^n double-precision complex amplitudes take $16 \cdot 2^n$ bytes—roughly 16 GB at $n = 30$ qubits—and every gate sweeps the entire array, so memory and time both scale as $\Theta(2^n)$, i.e. both grow in proportion to 2^n up to constant factors. It is this ceiling, not any cleverness in the algorithm, that decides how large a problem a given machine can handle, and it is where the 30-qubit limit of Section 4 comes from.

Other paradigms buy lower cost by giving up generality. Tensor-network and matrix-product-state methods [13, 14] lean on limited entanglement, stabilizer simulators handle Clifford circuits in polynomial time [15], and density-matrix methods represent mixed states but square the amplitude count to do it [12]. State-vector simulation is the most general of the lot and the easiest to reason about while learning, so it is the one we build on here.

2.5 Where QuantumSimEvo Fits

The mature state-vector simulators surveyed in Section 1 (Qiskit Aer, Cirq, ProjectQ, QuEST, Intel QS, cuQuantum) all optimize for peak throughput on a chosen target. QuantumSimEvo deliberately picks a different point on the trade-off: one dependency-light binary that runs correctly on a commodity PC and accelerates when it can, through the runtime backend selection of Section 3.

3 System Design and Architecture

In this section, the priorities laid out in Section 1 are turned into a concrete architecture. Here, these priorities are formulated as three explicit requirements: **(R1) portability**—the simulator must run correctly on any commodity machine and must never refuse to run because a hardware feature is absent; **(R2) separation of concerns**—circuit semantics must be decoupled from amplitude manipulation, so the user-facing model stays fixed while the execution engine changes beneath it; and **(R3) extensibility by addition**—a new execution backend must slot in without modifying code that already works. The remainder of the section shows how a layered design and runtime backend selection deliver all three from a single binary.

3.1 Layered Architecture

The simulator is implemented in C++ using only the standard library. C++ was chosen because it gives direct control over memory layout and zero-overhead access to the SIMD and CUDA facilities the accelerated backends rely on, while compiling to a single dependency-light binary—a direct fit for requirements R1 and R3.

As shown in Figure 1, the system is organized into four layers, each depending only on the one below it:

The **storage layer** (`StateVector`) owns the quantum state as a contiguous `std::vector<std::complex<double>>` of 2^n amplitudes. Alongside the high-level vector interface

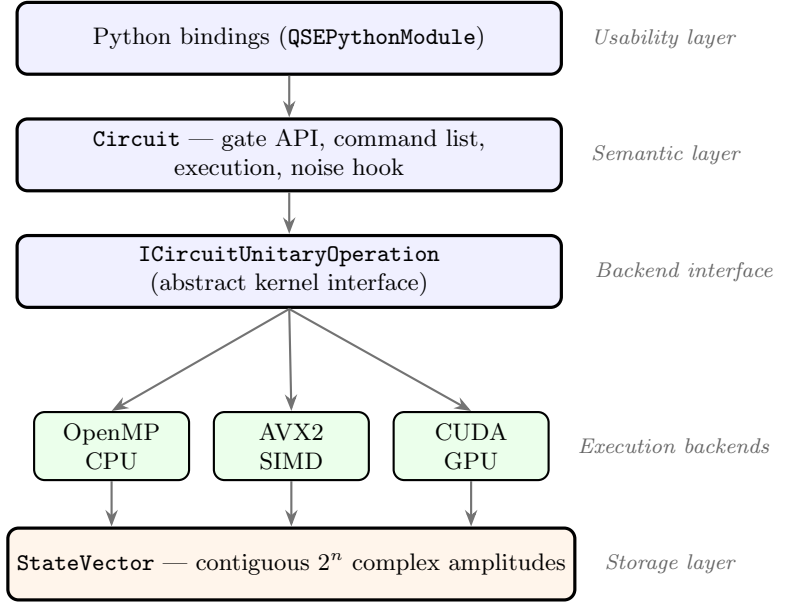


Figure 1: Layered architecture of QuantumSimEvo. Each layer depends only on the one below it. The abstract `ICircuitUnitaryOperation` interface decouples circuit semantics from execution, so backends are interchangeable and addable without changes to the layers above.

used by the CPU backends it also exposes a raw pointer accessor (`getRawPtr()`); that accessor is there so the GPU backend can map or transfer the buffer directly, without going through an extra abstraction.

The **semantic layer** (`Circuit`) is the user-facing API—`hadamard`, `cnot`, `rotateX`, and so on—and records the operations as a command list that gets replayed at `execute()` time. The important point is what `Circuit` deliberately omits: it holds no amplitude-manipulation logic of its own and hands every gate off to the backend it owns through a `std::unique_ptr<ICircuitUnitaryOperation>`.

The **backend interface** (`ICircuitUnitaryOperation`) is the keystone of the whole design; Section 3.2 covers it in detail.

The **usability layer** re-exposes the same circuit API to Python, so the simulator can be driven from

an interactive notebook—the natural place to teach from—with no recompilation.

3.2 The Backend Abstraction

All hardware-specific behavior is hidden behind a single pure-virtual interface, `ICircuitUnitaryOperation`, which declares one method per supported operation:

Each backend is a concrete implementation of this interface, and a circuit written once runs unchanged no matter which implementation is installed. That is exactly why the GPU backend counts as an *addition* and not a rewrite: it is one more subclass, invisible to `Circuit`, to the algorithms, and to the Python layer.

We chose to express the interface in terms of *quantum operations* (apply a Hadamard to qubit q) rather than generic linear-algebra primitives. That extra bit of abstraction lets each backend take advantage of a particular gate’s structure—a Pauli- X , for instance, just swaps amplitude pairs—while keeping the interface small enough that someone reading the code to learn the system can still follow it. Table 1 lists the full set of operations the interface declares.

The kernels all share one access pattern. Applying a single-qubit gate to qubit q splits the 2^n amplitudes into pairs a stride of 2^q apart and applies a 2×2 transformation to each pair. That regular, data-parallel shape is what makes the work a good fit for OpenMP threads, AVX2 lanes, and GPU thread blocks alike, and it is the reason a single interface can cover all three without hardware detail leaking upward.

3.3 Runtime Backend Selection

Since it is not known in advance what machine the simulator will run on, the backend is picked *at runtime* rather than at compile time. `CircuitUnitaryOperationFactory::create()` inspects the host and hands back the most capable implementation it can use:

A CUDA device is detected through the runtime device query, and AVX2 support through the `CPUID` instruction (leaf 7), so one compiled binary adapts to the machine in front of it: the GPU backend when a usable device is present, SIMD kernels on

a recent CPU, and a quiet fall-back to the portable OpenMP backend otherwise. This *graceful degradation* is where the portability goal actually shows up in the code—the user does not have to configure anything, and the program never refuses to run just because a hardware feature is missing.

Selection happens once, when a `Circuit` is constructed: the factory runs its capability probes a single time and reuses the chosen backend for every gate and every shot of that circuit. The probes themselves—a CUDA device query and a `CPUID` check—cost on the order of microseconds, so the decision is paid once at start-up and never repeated per simulation. The current backend tiers are summarized in Table 2.

The GPU tier is just one more branch in the factory, gated behind its own capability check (`supportsCUDA()`) in the same way as AVX2; adding it touched no other file.

3.4 Extensibility: Adding a Backend

The central claim of the design is that the execution engine can grow without disturbing the rest of the system. Adding a backend takes three local steps and nothing more:

1. **Implement** the 13 methods of `ICircuitUnitaryOperation` in a new class.
2. **Register** it in `CircuitUnitaryOperationFactory` behind an appropriate capability probe.
3. **Build**—no changes to `StateVector`, `Circuit`, the code implementing the quantum algorithms, or the Python bindings.

Adding a new gate works the same way: extend the interface, implement it in each backend, and expose it on `Circuit`. By the same token the GPU backend tests the architecture rather than altering it, which is exactly the property the design goals were after.

4 Algorithms and Case Studies

To show that the simulator is expressive enough for real workloads, and that the backend abstrac-

```

class ICircuitUnitaryOperation {
public:
    virtual void applyGate(StateVector& sv, const Gate2x2& gate, size_t q) = 0;
    virtual void applyCNOT(StateVector& sv, size_t control, size_t target) = 0;
    virtual void applyHadamard(StateVector& sv, size_t q) = 0;
    // ... 13 gate operations in total
};

```

Figure 2: The backend interface ICircuitUnitaryOperation.

Operation	Gate	Action
applyGate	generic U	Apply an arbitrary single-qubit 2×2 unitary
applyHadamard	H	Place the target qubit in an equal superposition
applyPauliX	X	Bit flip
applyPauliY	Y	Combined bit- and phase-flip
applyPauliZ	Z	Phase flip
applyPhase	$P(\theta)$	Apply a relative phase to $ 1\rangle$
applyRotateX	$R_x(\theta)$	Rotation about the x axis
applyRotateY	$R_y(\theta)$	Rotation about the y axis
applyRotateZ	$R_z(\theta)$	Rotation about the z axis
applyCNOT	CNOT	Flip the target iff the control qubit is set
applyCPhase	$CP(\theta)$	Controlled relative phase
applyMCPhase	$MCPPhase(\theta)$	Multi-controlled relative phase
applySwap	SWAP	Exchange the states of two qubits

Table 1: Gate operations declared by ICircuitUnitaryOperation and exposed on the Circuit API.

tion of Section 3.2 holds up under them, the validation suite outlined in Section 1 was built entirely on the public Circuit gate API—`hadamard`, `cnot`, `cphase`, `mcphase`, the Pauli gates (X , Y , Z), the rotations, and `swap`. None of the algorithm implementations access the state vector or the backends directly; each runs unchanged on whichever backend the factory picks at runtime. The two entanglement primitives (Bell, GHZ) serve as warm-up and correctness checks; the five algorithms that follow cover a transform (QFT [16]), an amplitude-amplification search (Grover [2]), a fault-relevant arithmetic routine (Shor [1]), and two variational methods [3, 11].

4.1 Bell and GHZ States

The two-qubit Bell state is the most basic example of entanglement, and it is the simulator’s first correctness check. The circuit is a Hadamard followed by

a CNOT, which leaves a maximally entangled pair whose only nonzero amplitudes sit on the two correlated basis states. To check the measurement machinery, the demo runs the circuit over many independent trials, collapsing and tallying the outcome each time, and confirms that the empirical distribution piles up on the expected correlated states in roughly equal proportion.

The GHZ state is the n -qubit version: a Hadamard on the first qubit and then a chain of CNOTs ties every qubit into the superposition $\frac{1}{\sqrt{2}}(|0\rangle^{\otimes n} + |1\rangle^{\otimes n})$. GHZ states are prepared on as many as 25 qubits, and it is verified that sampled measurements collapse to either the all-zeros or the all-ones string. That confirms correctness at scale and, at the same time, exercises the large-state-vector memory path.

```

std::unique_ptr<ICircuitUnitaryOperation>
CircuitUnitaryOperationFactory::create() {
    if (supportsCUDA())
        return std::make_unique<CircuitUnitaryOperationGPU>();
    else if (supportsAVX2())
        return std::make_unique<CircuitUnitaryOperationAVX>();
    else
        return std::make_unique<CircuitUnitaryOperation>(); // OpenMP CPU fallback
}

```

Figure 3: Runtime backend selection in `CircuitUnitaryOperationFactory::create()`.

Backend	Mechanism	Role
<code>CircuitUnitaryOperation</code>	OpenMP multithreading	Portable baseline
<code>CircuitUnitaryOperationAVX</code>	AVX2 SIMD + OpenMP	Default on modern x86
<code>CircuitUnitaryOperationGPU</code>	CUDA	Offload for large n

Table 2: Execution backends and their roles.

4.2 Quantum Fourier Transform

The QFT follows the standard construction [12, 16]: for each qubit i , a Hadamard followed by a ladder of controlled-phase rotations $\text{CP}(\pi/2^{j-i})$ from every higher qubit j , then a layer of swaps to do the bit reversal. The inverse transform reverses the bits first and negates each phase angle. We expose it as a reusable block (`createQFT` / `createIQFT` over an arbitrary qubit list), and Shor’s algorithm later reuses it verbatim. As a sanity check, the demo prepares a computational-basis input, applies the QFT, and samples the output.

4.3 Grover’s Search

For Grover’s algorithm [2], the driver sizes the register to the smallest n with $2^n > \text{target}$ and runs the optimal $\lfloor (\pi/4)\sqrt{2^n} \rfloor$ iterations [17]. This count assumes a single marked target ($M = 1$ among $N = 2^n$ states); for M solutions it generalises to $\lfloor (\pi/4)\sqrt{N/M} \rfloor$. Each iteration is a phase oracle followed by the diffusion operator. The oracle wraps a multi-controlled phase (`MCPPhase`(π), a multi-controlled Z) in X gates that pick out the target bit pattern, and the diffusion operator is the standard $H^{\otimes n} X^{\otimes n} \text{MCPPhase}(\pi) X^{\otimes n} H^{\otimes n}$. We read the answer out two ways—by sampling the amplified dis-

tribution and by one collapsing measurement—and the output of the demo indicates whether the marked state came back. Grover doubles as a good stress test: its multi-controlled phase drives the `MCPPhase` kernel that several of the other algorithms also rely on.

4.4 QAOA for MaxCut

The MaxCut QAOA [3] prepares a uniform superposition and then applies p alternating layers. In the cost layer, each graph edge (u, v) gets $e^{-i\gamma Z_u Z_v}$ via the usual $\text{CNOT}-R_z(2\gamma)-\text{CNOT}$ decomposition [12]; the mixer applies $R_x(2\beta)$ to every qubit. The angles γ, β are given per layer. To be able to check the quantum result, the module ships an OpenMP-parallel brute-force solver that enumerates every partition and reports the true maximum cut, and the most-sampled bitstring is compared to it.

4.5 Shor’s Algorithm

Shor’s algorithm [1] is the most demanding of the case studies. To factor an integer N of bit-length n (the number of bits needed to represent N) it uses a space-efficient $4n + 2$ qubit architecture that follows the Fourier-basis arithmetic of Beauregard [10]. We keep the full $2n$ -qubit phase register instead of

Beauregard’s single recycled control qubit: which requires $2n$ extra qubits but avoids mid-circuit measurement and classical feed-forward, leaving a purely unitary circuit that is simpler to run on a state vector. All the modular arithmetic happens in the Fourier basis [18, 10]. Constant addition becomes a layer of single-qubit phase rotations (`addConstantQFT`), and its singly- and doubly-controlled forms come from `cphase` and `mphase`. With these, a doubly-controlled modular adder (`ccModAdd`) is built. This in turn builds the controlled modular multiplier $U_a : |x\rangle \mapsto |ax \bmod N\rangle$ from controlled-SWAPs made of Toffoli gates—the doubly-controlled NOT (CCNOT), which flips its target qubit only when both control qubits are set—each synthesized as $H \text{MCPhase}(\pi) H$. The phase-estimation driver puts Hadamards on the $2n$ -qubit phase register, runs the controlled $U_a^{2^k}$ ladder and an inverse QFT, and samples the register. Classical post-processing then recovers the period by a continued-fraction expansion of the sampled phase and pulls out a factor via the greatest common divisor $\gcd(a^{r/2} \pm 1, N)$. The demonstration is bounded by the simulator’s 30-qubit ceiling: since the architecture needs $4n+2$ qubits, the constraint $4n+2 \leq 30$ gives $4n \leq 28$, hence $n \leq 7$ (as n is an integer) and therefore $N \leq 2^7 - 1 = 127$. This puts the exponential-memory boundary right in front of the user—a lesson in its own right.

4.6 Travelling Salesman via Constraint-Preserving QAOA

The last case study solves small TSP instances with a Quantum Alternating Operator Ansatz [11] on an n^2 -qubit one-hot (city $\times$ step) encoding [19]. We first turn the TSP cost function, expressed as a quadratic unconstrained binary optimization (QUBO) problem, into an Ising Hamiltonian ($H = \sum_i h_i Z_i + \sum_{i < j} J_{ij} Z_i Z_j$) with the substitution $x = (1 - Z)/2$ [19]. Instead of pushing every feasibility constraint into an energy penalty, the ansatz begins from a feasible permutation and uses an *XY ring mixer* [11, 20] within each city-row, $e^{-i\beta(X_i X_j + Y_i Y_j)}$. Because that mixer only hops one excitation around the ring, each city stays assigned to exactly one step

by construction. The reachable space shrinks from 2^{n^2} to n^n and the feasible-sample rate goes up, leaving only the per-step (column) constraint as a penalty in the phase separator. The R_{xx} and R_{yy} interactions reuse the same CNOT- R_z -CNOT core as Max-Cut, conjugated by Hadamard and $R_x(\pm\pi/2)$ basis changes. We decode the sampled bitstrings into tours, score them, and compare against a brute-force optimum. This is the deepest, most structured circuit in the suite, and it is still built only from the primitive gate set.

5 Results

This section evaluates QuantumSimEvo in two parts: first the wall-clock cost of running each algorithm in the validation suite on the reference hardware, then a direct head-to-head comparison against Qiskit Aer [4] on three representative workloads. All benchmarks use the hardware listed in Table 3.

5.1 Algorithm Runtimes

Table 4 reports wall-clock times for each algorithm at a representative problem size on the hardware described in Table 3. The most demanding workload, Shor’s factorisation of 115, runs for just under three hours on 30 qubits; the practical ceiling imposed by the $4n+2$ qubit requirement is a direct, tangible illustration of the exponential cost discussed in Section 2. The TSP case study makes the backend advantage concrete: the same 25-qubit XY-QAOA circuit completes in 64 seconds on the AVX2 CPU backend and in 12 seconds on the CUDA GPU backend, a $5.3\times$ speed-up from the GPU.

Figure 4 shows the per-gate average execution time for both the AVX2 and CUDA backends across the full range of 1 to 30 qubits (CPU) and 1 to 28 qubits (GPU). Four regions are visible. At very small register sizes (one to a few qubits) the per-gate time reflects fixed costs—thread or kernel launch, and on the GPU a host-device round trip—rather than useful work; this is why the GPU’s one-qubit point sits *above* its two-qubit point, and why both curves stay essentially flat here. Through the low-to-mid

Component	Specification
CPU	Intel Core i9-12900H
GPU	NVIDIA RTX 3070 Ti (8 GB)
RAM	DDR4 32 GB

Table 3: Hardware specification used for all benchmarks.

range the AVX2 CPU backend is fastest, its working set still resident in cache. Beyond ≈ 21 qubits the curves cross: the state vector outgrows the CPU cache, memory-bandwidth pressure dominates, and the GPU’s higher bandwidth takes the lead. In the final region both curves climb in proportion to 2^n , the expected memory-bound regime.

5.2 Comparison with Qiskit Aer

This comparison places QuantumSimEvo directly against Qiskit Aer [4] on three representative workloads—GHZ-state preparation, the Quantum Fourier Transform (QFT), and its inverse (IQFT)—each run with 1000 shots on the hardware of Table 3. Table 5 reports the CPU results and Table 6 the GPU results; all entries are wall-clock execution times, and for the millisecond-scale runs a warm-up execution is discarded so that one-time context- and kernel-initialisation costs are not charged to the measurement.

On the CPU (Table 5) the two simulators are closely matched. At 30 qubits and 1000 shots the GHZ benchmark takes 26 s on Qiskit Aer against 29 s on QuantumSimEvo, a difference of under 12% that is well within the range attributable to process scheduling and allocator behaviour across runs. The smaller circuits tell the same story: at 25 qubits GHZ completes in 540 ms versus 680 ms, the QFT in 3879 ms versus 3558 ms—here QuantumSimEvo is marginally *faster*—and the IQFT in 2292 ms versus 3539 ms. At 30 qubits both simulators sit in the two-minute range on the Fourier transforms and trade places within roughly 13%: QuantumSimEvo leads on the QFT (2:10 versus 2:29) while Aer leads on the IQFT (1:51 versus 2:05). Across every CPU workload the two stay within a small constant factor of one another.

On the GPU (Table 6) Qiskit Aer is consistently the faster of the two, but by a bounded, size-dependent margin. At 25 qubits Aer leads by roughly $3.7\times$ on GHZ, $2.6\times$ on QFT, and $1.7\times$ on IQFT; at 28 qubits the same comparisons fall to about $3.0\times$, $1.5\times$, and $1.2\times$. The gap therefore *narrows* as the state vector grows, and most sharply on the deeper Fourier circuits. This is consistent with Aer’s advantage being dominated by lower fixed per-execution overhead—kernel launches, transpilation, and host-device setup refined over years of production use, alongside NVIDIA’s cuStateVec kernels—rather than by a faster asymptotic amplitude sweep: as the qubit count rises that fixed cost is amortised over an exponentially larger array and the two backends converge toward the same memory-bandwidth-bound limit. The measured ratios do not reach unity within the 28-qubit ceiling the 8 GiB card allows; extrapolating the narrowing trend places parity only beyond that range, so on this hardware Qiskit Aer keeps a modest lead throughout the measured window.

The internal comparison between QuantumSimEvo’s own backends remains favourable. As Figure 4 shows, the GPU overtakes the AVX2 CPU backend beyond ≈ 21 qubits, where the state vector no longer fits in the CPU cache and memory-bandwidth pressure dominates, and the 25-qubit TSP circuit of Table 4 runs $5.3\times$ faster on the GPU than on the CPU. The crossover is sharp and predictable, which supports the runtime selection policy: defaulting to the GPU when CUDA is available is the right choice for any circuit large enough to benefit from it.

6 Discussion

The three priorities set out in Section 1—portability over peak performance, separation of circuit semantics from execution, and extensibility by addition—are reflected in what the system actually delivered. The runtime backend selector means a student on a laptop without a CUDA device gets a working simulator, not an error message; and the `ICircuitUnitaryOperation` interface meant the GPU backend was added without touching `Circuit`, the algorithms, or the Python bindings. The TSP

Algorithm	Problem Description	Qubits	Time (hh:mm:ss)
GHZ state	—	30	00:00:29
QFT	—	30	00:02:10
Inverse QFT	—	30	00:02:05
Grover (GPU)	Search of 29123321	25	01:22:30
QAOA (MaxCut)	30 nodes, 45 edges	30	00:04:38
Shor	Factorization of 115	30	02:49:35
TSP XY-QAOA (CPU)	5 cities	25	00:01:04
TSP XY-QAOA (GPU)	5 cities	25	00:00:12

Table 4: Wall-clock timing of the implemented algorithms.

Simulator	Algorithm	Qubits	Shots	Time
Qiskit Aer	GHZ	30	1000	00:26
Qiskit Aer	GHZ	25	1000	540 ms
QuantumSimEvo	GHZ	30	1000	00:29
QuantumSimEvo	GHZ	25	1000	680 ms
Qiskit Aer	QFT	25	1000	3879 ms
Qiskit Aer	QFT	30	1000	02:29
QuantumSimEvo	QFT	25	1000	3558 ms
QuantumSimEvo	QFT	30	1000	02:10
Qiskit Aer	IQFT	25	1000	2292 ms
Qiskit Aer	IQFT	30	1000	01:51
QuantumSimEvo	IQFT	25	1000	3539 ms
QuantumSimEvo	IQFT	30	1000	02:05

Table 5: CPU benchmark: QuantumSimEvo versus Qiskit Aer.

case study, the deepest circuit in the suite, ran correctly on all three backends from the same algorithm source, which is the clearest single test of whether the abstraction holds under a nontrivial workload.

Two findings stand out from the evaluation. First, on the CPU the abstraction that is the core of QuantumSimEvo’s design is effectively free: dispatching every gate through `ICircuitUnitaryOperation` to a backend chosen at runtime does not lead to extra measurable costs against a production simulator on the same machine. For an educational tool this is the more important of the two results, because the CPU backend is what a student without a CUDA device actually runs. Second, on the GPU a hand-written CUDA backend trails a mature, vendor-tuned one by a factor that is bounded and shrinks with problem size. Both outcomes are consistent with the same

underlying picture: state-vector simulation is fundamentally memory-bandwidth bound at the qubit counts of interest, so once a backend sweeps the amplitude array with enough parallelism to saturate the available bandwidth, differences between implementations are dominated by fixed per-execution overhead rather than by the inner kernels.

The results also clarify where the GPU backend gives up ground. The largest relative gaps appear on the smallest and shallowest circuits—largest of all for GHZ, a circuit of only n gates, and smallest for the deeper Fourier transforms—which is exactly where launch and setup overhead are least amortised. This points to per-gate launch cost, rather than per-gate arithmetic, as the most promising lever for future optimisation.

Several caveats bound these conclusions. All mea-

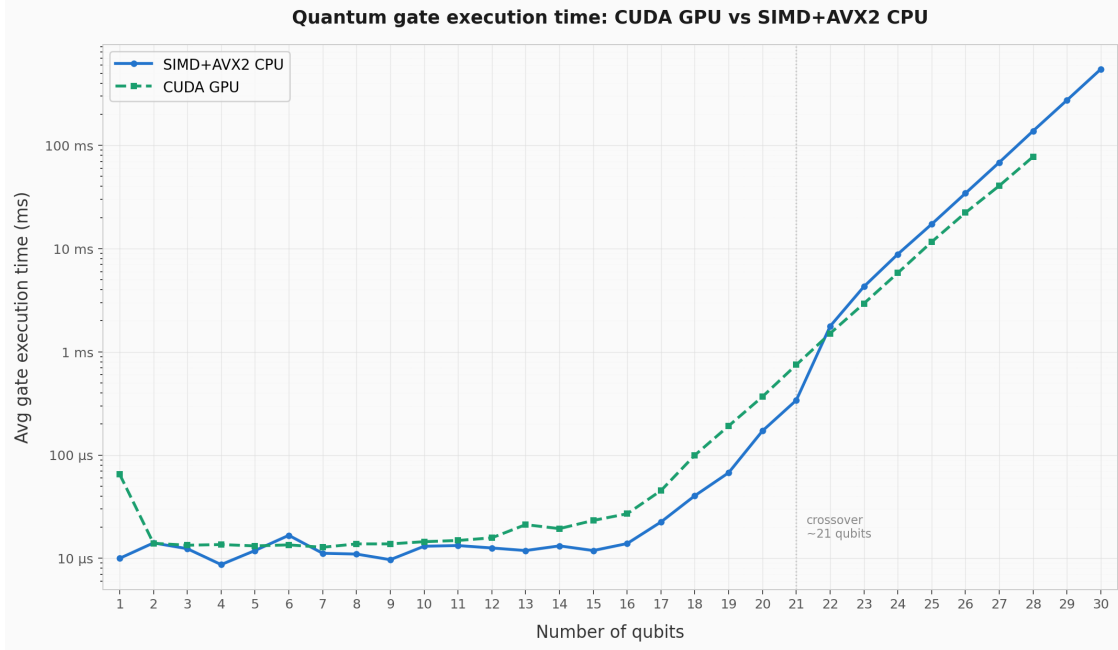


Figure 4: Average per-gate execution time vs. qubit count for the AVX2 CPU and CUDA GPU backends.

measurements come from a single machine (Table 3), so the absolute numbers reflect one CPU–GPU pairing rather than a trend established across hardware. The mobile RTX 3070 Ti is moreover subject to thermal and power limits that a desktop card would not impose, which can introduce run-to-run variation. Finally, the comparison fixes the shot count at 1000 and uses three small algorithm families; it characterises gate-application and sampling throughput, not the full range of circuit structures a simulator may encounter. We therefore report the data as an indication of competitiveness rather than as a precise ranking.

The design also points to several natural extensions, each an addition rather than a rewrite. A distributed-memory backend that splits the amplitude array across nodes, following the approach of QuEST [7] or the Intel Quantum Simulator [8], would slot in behind `ICircuitUnitaryOperation` without modifying existing circuit, algorithm, or binding code, and would push the qubit ceiling past the

single-node memory limit; a generic n -qubit unitary kernel would let users define custom gates beyond the built-in set; and connecting a classical optimiser in the outer loop of QAOA would turn the MaxCut and TSP demonstrations into end-to-end variational solvers. The comparison with Qiskit Aer also points to a concrete performance target: because the GPU gap is concentrated in fixed per-execution overhead rather than in the amplitude sweep itself, the most effective improvements would reduce kernel-launch and host–device transfer costs—batching gates into fused kernels, keeping the state vector resident on the device, and optionally dispatching to NVIDIA’s `cuStateVec` when present—each narrowing the remaining gap without compromising the portability and zero-dependency properties that motivate the project.

Finally, it is worth returning to the contribution that motivated the project. The performance results matter most because of who they are for: they show that the portability and zero-setup goals were not bought at the price of usability. A user can place one

Simulator	Algorithm	Qubits	Shots	Time
QuantumSimEvo	GHZ	25	1000	762 ms
QuantumSimEvo	GHZ	28	1000	5064 ms
QuantumSimEvo	QFT	25	1000	1053 ms
QuantumSimEvo	QFT	28	1000	6838 ms
QuantumSimEvo	IQFT	25	1000	1043 ms
QuantumSimEvo	IQFT	28	1000	6844 ms
Qiskit Aer	GHZ	25	1000	207 ms
Qiskit Aer	GHZ	28	1000	1682 ms
Qiskit Aer	QFT	25	1000	410 ms
Qiskit Aer	QFT	28	1000	4597 ms
Qiskit Aer	IQFT	25	1000	602 ms
Qiskit Aer	IQFT	28	1000	5534 ms

Table 6: GPU benchmark: QuantumSimEvo versus Qiskit Aer (RTX 3070 Ti).

dependency-light binary on an ordinary laptop, run the same code this evaluation benchmarks, and obtain production-comparable CPU performance—and a working GPU path where the hardware allows—without installing a framework, fighting a build, or owning special hardware. Lowering that barrier between the learner and the concepts, rather than topping any single performance chart, is the outcome we consider most significant.

7 Conclusion

We presented QuantumSimEvo, a portable C++ state-vector simulator designed for education and experimentation on commodity hardware. The central contribution is an architecture that separates quantum-circuit semantics from amplitude manipulation behind a single abstract interface, `ICircuitUnitaryOperation`, allowing execution backends to be swapped at runtime without touching circuit, algorithm, or Python-binding code. Three backends—a portable OpenMP baseline, an AVX2 SIMD-accelerated backend, and a CUDA GPU backend—are selected automatically at startup based on what the host machine supports, so the simulator runs correctly everywhere and accelerates wherever it can.

Across a suite of five algorithms and two entangle-

ment primitives, all implemented through the public gate API, the architecture held up: the same circuit code ran unchanged on every backend, with CPU performance comparable to a mature production framework and a working GPU path wherever the hardware allowed. By lowering the barrier between a learner and the concepts—rather than chasing peak throughput—QuantumSimEvo keeps quantum-circuit simulation, performance included, within reach on everyday hardware.

References

- [1] P. W. Shor, “Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer,” *SIAM J. Comput.*, vol. 26, no. 5, pp. 1484–1509, 1997.
- [2] L. K. Grover, “A fast quantum mechanical algorithm for database search,” in *Proc. 28th Annu. ACM Symp. Theory of Computing (STOC)*, 1996, pp. 212–219.
- [3] E. Farhi, J. Goldstone, and S. Gutmann, “A Quantum Approximate Optimization Algorithm,” arXiv:1411.4028, 2014.
- [4] Qiskit contributors, “Qiskit: An Open-Source Framework for Quantum Computing,” IBM

- Quantum, 2023. [Online]. Available: <https://www.qiskit.org>
- [5] Cirq Developers, “Cirq: A Python framework for creating, editing, and invoking Noisy Intermediate-Scale Quantum (NISQ) circuits,” Google Quantum AI, 2023. [Online]. Available: <https://quantumai.google/cirq>
 - [6] D. S. Steiger, T. Häner, and M. Troyer, “ProjectQ: An Open Source Software Framework for Quantum Computing,” *Quantum*, vol. 2, p. 49, 2018.
 - [7] T. Jones, A. Brown, I. Bush, and S. C. Benjamin, “QuEST and High Performance Simulation of Quantum Computers,” *Scientific Reports*, vol. 9, no. 1, p. 10736, 2019.
 - [8] M. Smelyanskiy, N. P. D. Sawaya, and A. Aspuru-Guzik, “qHiPSTER: The Quantum High Performance Software Testing Environment,” arXiv:1601.07195, 2016.
 - [9] H. Bayraktar *et al.*, “cuQuantum SDK: A High-Performance Library for Accelerating Quantum Science,” in *Proc. IEEE Int. Conf. Quantum Computing and Engineering (QCE)*, 2023, pp. 1050–1061.
 - [10] S. Beauregard, “Circuit for Shor’s algorithm using $2n + 3$ qubits,” *Quantum Information and Computation*, vol. 3, no. 2, pp. 175–185, 2003.
 - [11] S. Hadfield, Z. Wang, B. O’Gorman, E. G. Rieffel, D. Venturelli, and R. Biswas, “From the Quantum Approximate Optimization Algorithm to a Quantum Alternating Operator Ansatz,” *Algorithms*, vol. 12, no. 2, p. 34, 2019.
 - [12] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, 10th Anniversary ed. Cambridge, U.K.: Cambridge Univ. Press, 2010.
 - [13] G. Vidal, “Efficient classical simulation of slightly entangled quantum computations,” *Phys. Rev. Lett.*, vol. 91, no. 14, p. 147902, 2003.
 - [14] I. L. Markov and Y. Shi, “Simulating quantum computation by contracting tensor networks,” *SIAM J. Comput.*, vol. 38, no. 3, pp. 963–981, 2008.
 - [15] S. Aaronson and D. Gottesman, “Improved simulation of stabilizer circuits,” *Phys. Rev. A*, vol. 70, no. 5, p. 052328, 2004.
 - [16] D. Coppersmith, “An approximate Fourier transform useful in quantum factoring,” IBM Research Report RC 19642, 1994. arXiv:quant-ph/0201067, 2002.
 - [17] M. Boyer, G. Brassard, P. Høyer, and A. Tapp, “Tight bounds on quantum searching,” *Fortschritte der Physik*, vol. 46, no. 4–5, pp. 493–505, 1998.
 - [18] T. G. Draper, “Addition on a quantum computer,” arXiv:quant-ph/0008033, 2000.
 - [19] A. Lucas, “Ising formulations of many NP problems,” *Frontiers in Physics*, vol. 2, p. 5, 2014.
 - [20] Z. Wang, N. C. Rubin, J. M. Dominy, and E. G. Rieffel, “XY mixers: Analytical and numerical results for the quantum alternating operator ansatz,” *Physical Review A*, vol. 101, no. 1, p. 012320, 2020.